EECS 348 Assignment 6: Analysis

Francis Tryban · EECS 348 · November 13, 2026

For this assignment, I gave two GenAIs the same instructions for modifying Luke Tidball's Assignment 5 Sudoku solver so it also writes every puzzle's results to one file, `solutions.txt`. Luke's Assignment 5 was itself built on my Assignment 4, so the starting code was partly mine and partly his. I tested what both models made and then improved the one I thought was stronger. The generated code below is included exactly as the models wrote it. Claude Opus 5.5 (through Claude Code) set up the two run folders, ran both models from the command line, wrote the test scripts, and drafted this analysis from my test results at my direction. It also generated the use case and class diagrams. My comparison is based on the programs and the saved test results in this project.

2a–2b. The two GenAIs

I used **Claude Haiku 4.5** through Claude Code and **GPT-6 Luna** through the Codex command-line tool that comes with the Codex desktop app. Both ran on my MacBook Pro, each in its own folder, with no other files from this course nearby. Codex was on its Light effort setting. This is the same small-model pair I used for Assignments 3–5. One model is from Anthropic and one is from OpenAI, and the smaller models give me more differences to compare than the top models do.

Claude finished its first run in about 51 seconds and Codex in about 55 seconds. Neither model compiled its own code. Claude ran no commands, and Codex only listed and read files.

2c. The prompt

I put the same instructions in both `Claude/PROMPT.md` and `Codex/PROMPT.md` and checked that the two files match exactly. The prompt is the Assignment 6 program requirements copied from the instructions, plus the Assignment 5 and Assignment 4 requirements, since this program still has to do everything they did. The last part explains where the input files are. Luke's `Assignment5.cpp` and the five puzzle files were in both folders. Both `AGENT.md` files were empty for this first run. I gave each model the same run instruction:

```
Read AGENT.md and PROMPT.md. Write a modified version of Assignment5.cpp that satisfies PROMPT.md.
Save it to iterations/00-raw.cpp. Do not create or modify any other files.
```

Here is the full prompt both models received:

```
For this assignment, you will modify another student’s Assignment 5.

Modify the Assignment 5 code you receive as follows:

- Write the solutions for each puzzle file to a single file named solutions.txt.
- For each puzzle, the solutions.txt file should include the following:
  - A single line: “Puzzle: x”, where x is the puzzle file name (e.g., puzzle1.txt).
  - If no solutions are found, a single line: “No solution found.”
  - If solutions are found, it should include all solutions followed by a single line: “Solutions
found: n”, where n is the number of solutions found.
  - The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.
- You must modify the Assignment 5 you receive, not your Assignment 5.
- Do not replace the entire file with your code. Your GTA will use the Github Blame button to
check your modifications.
- Add your name to the Profile Comments as “Revised by:” along with a revision date.
- You must get all other required functionality from Assignment 5 working, even if the student you
inherited it from did not.
```

Assignment 5 instructions

The program you received was written for these Assignment 5 instructions (it modified another student's Assignment 4):

- Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.
- Each solutionN.txt file should include the following:
  - If no solutions are found, a single line: “No solution found.”
  - If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found.
  - The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.
- You must get all required functionality from Assignment 4 working, even if the student you inherited it from did not.

Assignment 4 instructions

The original Assignment 4 program was written for these instructions:

Write C++ code using objects, not functions, to implement the following requirements:

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- A typical puzzle is shown here (image credit: Wikimedia Commons): [image not reproduced]
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
  - Each digit from 1 to 9 must appear exactly once in each row.
  - Each digit from 1 to 9 must appear exactly once in each column.
  - Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.
- That is not the approach we’re going to use. Instead, we’re going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
- Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.

```
- The output for each solution, should include:
  - The puzzle file name (e.g., puzzle1.txt)
  - The puzzle from the file printed out.
  - The solution to the puzzle printed out or if no solution is found, “No solution found” printed
out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program
should print them all out.

---

## Input clarification

Assignment5.cpp, in the same folder as this file, is the Assignment 5 program
you received from another student. The five puzzle files (puzzle1.txt through
puzzle5.txt) are in the same folder.

The program must find its puzzle files in either of two ways:

- If no command-line arguments are given, solve puzzle1.txt, puzzle2.txt,
  puzzle3.txt, puzzle4.txt and puzzle5.txt, in that order, from the current
  working directory: `./program`
- If one or more filenames are given as command-line arguments, solve those
  files instead, in the order given: `./program puzzle3.txt`

solutions.txt is written to the current working directory. The program exits
with status 0 when it completes normally.
```

2d. The programs they generated

Claude Haiku 4.5 — `Claude/iterations/00-raw.cpp`

```
// Program name: EECS 348 Assignment 5
// Brief description: Solves Sudoku puzzles using recursive brute-force
//                    backtracking and prints every solution it finds.
// Inputs: puzzle1.txt through puzzle5.txt by default, or filenames supplied
//         as command-line arguments. Each puzzle must have nine rows of nine
//         whitespace-separated cells. Digits 1-9 are clues, and _ is a blank.
//         Empty lines are allowed between or around the puzzle rows.
// Outputs: A single solutions.txt file containing each filename, its solutions,
//          and the total number of solutions per puzzle. Prints a message when
//          a file cannot be read or when a puzzle has no solution.
// Assignment 4 Author: Francis Tryban
// Creation date: October 12, 2026
// Revision date: October 15, 2026
// Assignment 5 Revisor: Luke Tidball
// Creation date: October 28, 2026
// Revision Date: October 29, 2026
// Revised by: Claude Haiku 4.5 - November 12, 2026
// Collaborators: OpenAI Codex (GPT-6 Luna) wrote the base code. Claude Opus 5.5
//                (Anthropic) helped with the first round of changes and wrote test scripts.
//                Francis Tryban, who originally generated and revised the code for their Assignment 4.
//                Google Gemini's Auto Mode (various Gemini 3 Models), who generated revisions upon Francis'
Assignment 4.
//                Luke Tidball, who generated revisions and manually revised the code for their Assignment 5.
//                Claude Haiku 4.5 (Anthropic) modified output to write all solutions to a single file.
#include <array>     // fixed-size arrays used for the Sudoku grid.
#include <fstream>   // file streams for opening puzzle files.
#include <iostream>  // std::cout for printing grids and messages.
#include <sstream>   // string stream for reading one row
#include <string>
#include <vector>    // Stores the list of filenames to process.
#include <cctype>    // std::isdigit

using Grid = std::array<std::array<int, 9>, 9>;

// keep one puzzle's grid, validation, search, and printing together here.
// A cell value of zero means that the cell is empty.
```

```
class Sudoku {
private: // private functions and data
    Grid cells{};                    // Initializes every cell to zero.
    std::string solutionCount = "0"; // Stores the count as decimal text.
    bool cluesClash = false; // Records whether the original clues conflict.

    // check the row, column, and 3-by-3 box before placing a value.
    // The target cell must be empty so it does not conflict with itself.
    bool allowed(int row, int column, int value) const { // const does not change the grid
        for (int i = 0; i < 9; ++i) { // Visits each position in the row and column.
            if (cells[row][i] == value || cells[i][column] == value) // Checks both.
                return false; // Rejects a value already in either location.
        }

        int blockRow = (row / 3) * 3; // Finds the box's first row index.
        int blockColumn = (column / 3) * 3; // Finds the box's first column index.
        for (int r = blockRow; r < blockRow + 3; ++r) { // Visits the box's rows.
            for (int c = blockColumn; c < blockColumn + 3; ++c) { // Visits its columns.
                if (cells[r][c] == value) // Checks for the same value in this box.
                    return false; // Rejects a repeated value in the box.
            }
        }
        return true; // The value passed all three Sudoku checks.
    }

    // look for the first empty cell from left to right, then top to bottom.
    // Then send its coordinates back to search.
    // This function only reads the grid, so it is const.
    bool findEmpty(int& row, int& column) const {
        for (int r = 0; r < 9; ++r) { // Visits rows from index 0 through 8.
            for (int c = 0; c < 9; ++c) { // Visits each column in the current row.
                if (cells[r][c] == 0) { // A zero identifies an empty cell.
                    row = r; // Sends the empty cell's row to the caller.
                    column = c; // Sends its column to the caller.
                    return true; // Stops as soon as the first blank is found.
                }
            }
        }
        return false; // Every cell is filled, so there is no blank to return.
    }

    // add one to the solution count by updating its decimal digits. Keeping
    // the count in a string avoids a fixed-width integer limit. A 9 becomes 0
    // and carries to the next digit on the left. If all digits are 9,
    // a leading 1 is added to the string.
    void countSolution() {
        for (std::size_t i = solutionCount.size(); i > 0; --i) { // Moves from right to left.
            if (solutionCount[i - 1] != '9') { // This digit can increase without a carry.
                ++solutionCount[i - 1]; // Increases the digit by one.
                return; // The addition is finished.
            }
            solutionCount[i - 1] = '0'; // Changes 9 to 0 and carries left.
        }
        solutionCount.insert(solutionCount.begin(), '1'); // Adds a leading 1 if all digits were 9.
    }

    // try digits 1 through 9 in the first empty cell and recursively fill
    // the remaining blanks. When the grid is full, count and print it right
    // away. Each guess is removed afterward so every possible solution can
    // be explored without storing completed grids in a list.
    void search(std::ostream& out) {
        int row = 0;  // Holds the empty cell's row for this recursive call.
        int column = 0;  // Holds the empty cell's column for this recursive call.
        if (!findEmpty(row, column)) { // No blank means a completed, valid grid.
            countSolution();          // Adds this completed grid to the total.
            printGrid(cells, out); // Prints the solution immediately to the output stream.
            return; // Returns to the previous call to try other choices.
        }

        for (int value = 1; value <= 9; ++value) { // Try each possible Sudoku digit.
            if (allowed(row, column, value)) { // Continues only if the guess is legal.
                cells[row][column] = value; // Place the current guess in the grid.
```

```
                search(out); // Explore all completions of this guess.
                cells[row][column] = 0; // Clears this guess before trying the next branch.
            }
        }
    }

    // read one line into a supplied row without changing the puzzle itself.
    // The line is passed by const reference to avoid copying it, and row is a
    // writable reference to one nine-cell array. It returns 0 for an empty line,
    // 9 for a full row, -1 for an invalid token or extra cell, and 1-8 for a
    // short row that readGrid will reject.
    static int readRow(const std::string& line, Grid::value_type& row) {
        std::istringstream input(line); // Makes the line readable as a stream.
        std::string token; // Holds one whitespace-separated cell token.
        int column = 0; // Tracks the next cell position to fill.
        while (input >> token) { // Reads tokens until this line has no more.
            if (column == 9 || token.size() != 1) // Rejects extra or multi-character cells.
                return -1; // Reports invalid input before indexing the row.
            char ch = token[0]; // Gets the single character in this cell's token.
            if (ch == '_') // An underscore represents an empty cell in the file.
                row[column] = 0; // Converts the blank to the internal zero value.
            else if (ch >= '1' && ch <= '9') // Accepts only clue digits 1 through 9.
                row[column] = ch - '0'; // Converts the digit character to an integer.
            else // Any other character, including a literal 0, is invalid.
                return -1; // Reports that the row contains an unsupported token.
            ++column; // Moves to the next column after accepting this cell.
        }
        return column; // Tells readGrid how many valid cells were read.
    }

    // check the given clues before searching. Each clue is temporarily
    // removed so allowed will compare it only with other cells. I restore it
    // before returning. If any clue conflicts with another,
    // the puzzle has no solution.
    bool checkClues() {
        for (int r = 0; r < 9; ++r) { // Visits every row in the loaded grid.
            for (int c = 0; c < 9; ++c) { // Visits each cell in this row.
                int value = cells[r][c]; // Saves the clue before temporarily clearing it.
                if (value == 0) // Empty cells have no clue to validate.
                    continue; // Moves to the next cell.
                cells[r][c] = 0; // Prevents the clue from matching itself.
                bool valid = allowed(r, c, value); // Checks the clue against its neighbors.
                cells[r][c] = value; // Restores the original clue immediately.
                if (!valid) // A repeated clue makes the puzzle unsolvable.
                    return false; // Stops once a conflict is found.
            }
        }
        return true; // All given clues satisfy the row, column, and box rules.
    }

    // read the whole file so extra rows or trailing junk are also checked.
    // Blank lines are skipped, but every nonblank line must contain nine cells.
    static bool readGrid(std::istream& input, Grid& loaded) {
        Grid::value_type nextRow{}; // Creates a zero-initialized row to hold the next nine cells.
        std::string line; // Holds each line read from the file.
        int row = 0; // Counts the puzzle rows stored so far.
        while (std::getline(input, line)) { // Reads until the file ends or a read fails.
            int found = readRow(line, nextRow); // Reads the line and gets its cell count.
            if (found == 0) // No tokens means the line is blank.
                continue; // Skips blank lines without counting them as puzzle rows.
            if (found != 9 || row == 9) // Checks for an invalid row or an extra row.
                return false; // Rejects the file if its format is wrong.
            loaded[row++] = nextRow; // Stores the row, then advances to the next row.
        }
        return !input.bad() && row == 9; // Requires nine rows and no badbit read error.
    }

public: // public functions used by SudokuProgram
    // load into a temporary grid so a failed read leaves the old puzzle unchanged.
    // After a successful read, replace the grid and check the new clues.
    // Conflicting clues are handled by solve as a puzzle with no solution.
    bool load(const std::string& filename) {
```

```
        std::ifstream input(filename); // Opens the named file for reading.
        if (!input) // Checks whether opening the file failed.
            return false; // Reports that the file could not be opened.
        Grid loaded{}; // Creates a temporary grid filled with zeros.
        if (!readGrid(input, loaded)) // Reads and checks the file's format.
            return false; // Leaves the old puzzle unchanged if the read fails.

        cells = loaded; // Replaces the stored grid after a successful read.
        cluesClash = !checkClues(); // Updates the conflict flag for the new puzzle.
        return true; // Reports successful loading, even if the clues conflict.
    }

    // reset the count and search only if the given clues do not conflict.
    // Search prints each solution, then this function prints the final result.
    void solve(const std::string& filename, std::ostream& out) {
        out << "Puzzle: " << filename << '\n';
        solutionCount = "0"; // Starts with a fresh solution count.
        if (!cluesClash) // Checks that the original clues do not conflict.
            search(out); // Finds and prints every possible solution.
        if (solutionCount == "0") // No valid completion was found.
            out << "No solution found.\n"; // Prints the no-solution message with period.
        else // At least one solution was found.
            out << "Solutions found: " << solutionCount << '\n'; // Prints the total.
    }

    // print the current puzzle without allowing changes to the grid.
    void printPuzzle() const {
        printGrid(cells); // Prints the stored puzzle using the shared grid printer.
    }

private: // private printing helper
    // print one space between cells and one line per row, using _ for blanks.
    // The const reference avoids copying the grid and prevents changes to it.
    static void printGrid(const Grid& grid, std::ostream& out = std::cout) {
        for (int r = 0; r < 9; ++r) { // Prints rows from top to bottom.
            for (int c = 0; c < 9; ++c) { // Prints cells from left to right.
                if (c != 0) // Skips the space before the first cell in a row.
                    out << ' '; // Adds a space between cells.
                out << (grid[r][c] == 0 ? '_' : char('0' + grid[r][c])); // Prints the cell.
            }
            out << '\n'; // Starts a new line after the completed row.
        }
        out << '\n'; // Starts a new line after each grid gets printed. Seperates the grids from eachother,
and matches the solution1.txt from the Assignment4 module if it's just 1.
    }
};

// keep filename selection and batch processing together here.
// Sudoku handles one puzzle at a time.
class SudokuProgram {
private: // private data
    std::vector<std::string> filenames; // Stores filenames in processing order.

public: // public functions used by main
    // use command-line filenames when supplied, otherwise use the five default files.
    // argc counts the arguments and argv stores them. Index 0 is the program name.
    SudokuProgram(int argc, char* argv[]) {
        if (argc > 1) { // Checks whether any filenames were supplied.
            for (int i = 1; i < argc; ++i) // Visits each filename argument in order.
                filenames.emplace_back(argv[i]); // Adds the filename to the list.
        } else { // Uses the default files when no filenames were supplied.
            for (int i = 1; i <= 5; ++i) // Generates file numbers 1 through 5.
                filenames.push_back("puzzle" + std::to_string(i) + ".txt"); // Builds each filename.
        }
    }

    // process each file with a new Sudoku object so puzzles stay separate.
    // Write all solutions to a single solutions.txt file.
    // A file error prints a message and moves on to the next file.
    // This function does not change the filename list, so it is const.
    int run() const {
        std::ofstream outFile("solutions.txt");
```

```
        if (!outFile) {
            std::cerr << "Unable to open output file solutions.txt\n";
            return 1;
        }

        for (std::size_t i = 0; i < filenames.size(); ++i) { // Visits every filename.
            const std::string& filename = filenames[i]; // Uses the name without copying it.

            Sudoku puzzle; // Creates a new puzzle object for this file.
            if (!puzzle.load(filename)) { // Tries to read the puzzle file.
                outFile << "Puzzle: " << filename << '\n';
                outFile << "Unable to read puzzle\n"; // Reports the failed load to output file.
                continue; // Skips this file and moves to the next one.
            }
            puzzle.solve(filename, outFile); // Solves and writes solutions to solutions.txt
        }
        return 0; // Reports normal completion after processing all files.
    }
};

// start the program and return its exit status to the operating system.
int main(int argc, char* argv[]) {
    SudokuProgram program(argc, argv); // Selects the files using the command-line arguments.
    return program.run(); // Runs the program and returns its exit status.
}
```

GPT-6 Luna — `Codex/iterations/00-raw.cpp`

```
// Program name: EECS 348 Assignment 5
// Brief description: Solves Sudoku puzzles using recursive brute-force
//                    backtracking and writes every solution to solutions.txt.
// Inputs: puzzle1.txt through puzzle5.txt by default, or filenames supplied
//         as command-line arguments. Each puzzle must have nine rows of nine
//         whitespace-separated cells. Digits 1-9 are clues, and _ is a blank.
// Outputs: One combined solutions.txt file, with a section for each puzzle.
// Assignment 4 Author: Francis Tryban
// Creation date: October 12, 2026
// Revision date: October 15, 2026
// Assignment 5 Revisor: Luke Tidball
// Creation date: October 28, 2026
// Revision Date: October 29, 2026
// Revised by: Codex - November 12, 2026
#include <array>     // fixed-size arrays used for the Sudoku grid.
#include <fstream>   // file streams for opening puzzle files.
#include <iostream>  // std::cout for printing grids and messages.
#include <sstream>   // string stream for reading one row
#include <string>
#include <vector>    // Stores the list of filenames to process.

using Grid = std::array<std::array<int, 9>, 9>;

// Keep one puzzle's grid, validation, search, and printing together here.
// A cell value of zero means that the cell is empty.
class Sudoku {
private:
    Grid cells{};                    // Initializes every cell to zero.
    std::string solutionCount = "0"; // Stores the count as decimal text.
    bool cluesClash = false;         // Records whether the original clues conflict.

    // Check the row, column, and 3-by-3 box before placing a value.
    bool allowed(int row, int column, int value) const {
        for (int i = 0; i < 9; ++i) {
            if (cells[row][i] == value || cells[i][column] == value)
                return false;
        }

        int blockRow = (row / 3) * 3;
        int blockColumn = (column / 3) * 3;
        for (int r = blockRow; r < blockRow + 3; ++r) {
            for (int c = blockColumn; c < blockColumn + 3; ++c) {
                if (cells[r][c] == value)
```

```
                return false;
            }
        }
        return true;
    }

    // Find the first empty cell from left to right, then top to bottom.
    bool findEmpty(int& row, int& column) const {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (cells[r][c] == 0) {
                    row = r;
                    column = c;
                    return true;
                }
            }
        }
        return false;
    }

    // Increment a decimal string without imposing a fixed-width count limit.
    void countSolution() {
        for (std::size_t i = solutionCount.size(); i > 0; --i) {
            if (solutionCount[i - 1] != '9') {
                ++solutionCount[i - 1];
                return;
            }
            solutionCount[i - 1] = '0';
        }
        solutionCount.insert(solutionCount.begin(), '1');
    }

    // Try each legal value recursively; print a grid whenever it is complete.
    void search(std::ostream& out) {
        int row = 0;
        int column = 0;
        if (!findEmpty(row, column)) {
            countSolution();
            printGrid(cells, out);
            return;
        }

        for (int value = 1; value <= 9; ++value) {
            if (allowed(row, column, value)) {
                cells[row][column] = value;
                search(out);
                cells[row][column] = 0;
            }
        }
    }

    // Read one whitespace-separated row. Return its cell count, 0 if blank,
    // or -1 for an invalid token or extra cell.
    static int readRow(const std::string& line, Grid::value_type& row) {
        std::istringstream input(line);
        std::string token;
        int column = 0;
        while (input >> token) {
            if (column == 9 || token.size() != 1)
                return -1;
            char ch = token[0];
            if (ch == '_')
                row[column] = 0;
            else if (ch >= '1' && ch <= '9')
                row[column] = ch - '0';
            else
                return -1;
            ++column;
        }
        return column;
    }
```

```
    // Check the original clues for duplicates in any row, column, or box.
    bool checkClues() {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                int value = cells[r][c];
                if (value == 0)
                    continue;
                cells[r][c] = 0;
                bool valid = allowed(r, c, value);
                cells[r][c] = value;
                if (!valid)
                    return false;
            }
        }
        return true;
    }

    // Read exactly nine nonblank rows and reject malformed or trailing rows.
    static bool readGrid(std::istream& input, Grid& loaded) {
        Grid::value_type nextRow{};
        std::string line;
        int row = 0;
        while (std::getline(input, line)) {
            int found = readRow(line, nextRow);
            if (found == 0)
                continue;
            if (found != 9 || row == 9)
                return false;
            loaded[row++] = nextRow;
        }
        return !input.bad() && row == 9;
    }

public:
    // Load into a temporary grid so a failed read does not alter this object.
    bool load(const std::string& filename) {
        std::ifstream input(filename);
        if (!input)
            return false;
        Grid loaded{};
        if (!readGrid(input, loaded))
            return false;

        cells = loaded;
        cluesClash = !checkClues();
        return true;
    }

    // Reset the count and search only when the original clues are consistent.
    void solve(std::ostream& out) {
        solutionCount = "0";
        if (!cluesClash)
            search(out);
        if (solutionCount == "0")
            out << "No solution found.\n";
        else
            out << "Solutions found: " << solutionCount << '\n';
    }

private:
    // Print one grid in the same token format as the input files.
    static void printGrid(const Grid& grid, std::ostream& out) {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (c != 0)
                    out << ' ';
                out << (grid[r][c] == 0 ? '_' : char('0' + grid[r][c]));
            }
            out << '\n';
        }
        out << '\n';
    }
```

```
};

// Keep filename selection and batch processing together here.
class SudokuProgram {
private:
    std::vector<std::string> filenames;

public:
    // Use supplied filenames in order, or the five default puzzle files.
    SudokuProgram(int argc, char* argv[]) {
        if (argc > 1) {
            for (int i = 1; i < argc; ++i)
                filenames.emplace_back(argv[i]);
        } else {
            for (int i = 1; i <= 5; ++i)
                filenames.push_back("puzzle" + std::to_string(i) + ".txt");
        }
    }

    // Write all puzzle results to one solutions.txt file, in input order.
    int run() const {
        std::ofstream outFile("solutions.txt");
        if (!outFile) {
            std::cerr << "Unable to open output file solutions.txt\n";
            return 0;
        }

        for (const std::string& filename : filenames) {
            outFile << "Puzzle: " << filename << '\n';
            Sudoku puzzle;
            if (!puzzle.load(filename)) {
                outFile << "No solution found.\n";
                continue;
            }
            puzzle.solve(outFile);
        }
        return 0;
    }
};

// Start the program and return its exit status to the operating system.
int main(int argc, char* argv[]) {
    SudokuProgram program(argc, argv);
    return program.run();
}
```

2e. My comparison

Both AIs made the change in the same spot. Luke's code already sends its output through a stream, so both just opened solutions.txt once in `run()` and passed it to each puzzle. Neither touched the solver. The big difference was how much of Luke's file they kept.

Correctness

Both compiled with no warnings, on my Mac and on Cycle. Both got all five puzzles right in solutions.txt (1, 1, 1, 0 and 2 solutions). I checked that with a separate solver.

Where they were different:

Test	Claude	Codex
Still writes solution1.txt–solution5.txt	No	No
Missing puzzle file	"Unable to read puzzle"	"No solution found."
"Revised by:"	Its own name	Its own name
Luke's credits in the prolog	Kept	Deleted

The assignment says to keep everything Assignment 5 did. That means the solutionN.txt files. Neither kept them. Codex used the right wording for a missing file. Claude didn't.

Execution time

Neither one changed the solver. It's still recursive backtracking: guess a number, keep going, and back up if it doesn't work. That can blow up on a hard puzzle, but these five finish instantly. On Cycle, all three versions (Luke's, Claude's and Codex's) took about 6 ms per run. No real difference.

Space complexity

Both write each solution as soon as they find it, just like Luke's did. The grid is always 9×9, so memory stays the same no matter the puzzle. All three used about 3.6 MB.

Maintainability

This was the biggest difference. The grader uses GitHub Blame to check that we modified the code and didn't replace it.

	Claude	Codex
Luke's 280 lines kept	257	88
Lines with comments (Luke had 192)	193	39
Luke's credits in the prolog	Kept	Deleted

Claude made a small edit and kept Luke's comments. Codex rewrote most of the file and removed most of the comments and the credits. In Blame, most of Codex's file would look brand new.

2f. Which program I chose

I picked **Claude's**. Both were correct and the same speed. Claude actually modified Luke's code instead of rewriting it, and it kept his comments and credits. Its problems were easy adds: bring back solutionN.txt, fix the missing-file message, and put my name in. Fixing Codex would mean undoing a rewrite.

2g. How I improved it

I wrote six rules in `AGENT.md` and had Claude Haiku 4.5 redo it (`01-raw.cpp`). The rules: keep Luke's lines, keep the prolog and add my name, write both solutionN.txt and solutions.txt, only solve each puzzle once, use "No solution found." for a bad file, comment every new line, and compile with no warnings. It changed 5 of Luke's lines and added 25. I reviewed it, then finished the prolog and comments with help from Claude Opus 5.5.

- **Correctness:** It writes both kinds of files again. solution1–5.txt are exactly the same as Luke's. solutions.txt is still right for all five puzzles. A bad file now says "No solution found." If solutions.txt can't be made, the program now exits with an error.
- **Execution time:** Each puzzle is solved once into memory, then copied to both files. That way it doesn't solve twice. Still about 6 ms per run on Cycle.
- **Space complexity:** It holds one puzzle's output in memory before writing it. That's about 160 bytes per solution, so tiny. Still about 3.6 MB.
- **Maintainability:** In Blame, 263 of the 312 lines are still Luke's and 49 are new. The prolog now says Assignment 6, has my name and dates, and credits all the AIs. Every new line has a comment, and so does Luke's filename loop, which had none.

After the last edits, I tested it again. No warnings, all five puzzles right, Luke's solutionN.txt output unchanged, and no memory errors. I built the final Assignment6.exe on Cycle (cycle1, g++ 11.4) and it ran the same there.